Project Title: AI-Augmented SQL Query Validator & Formatter

Compiler Design Lab Project Report

Team Members:

1. Gaurav Gopichand Nanhe - 24BCB0020
2. V Surya - 24BAI0351

1. Abstract

This report describes the design and implementation of an AI-augmented SQL query validator and formatter. The system accepts an SQL query, either entered directly or generated from a natural-language question by a large language model (LLM) and processes it through a classical, hand-built compiler pipeline: a lexical analyzer (lexer), a recursive-descent syntax analyzer (parser), a symbol-table-driven semantic validator, and an Abstract Syntax Tree (AST) pretty-printer (formatter).

Two decoupled AI components interface at the system boundaries:

1. An input-edge translator mapping natural-language queries into candidate SQL expressions.
2. An output-edge diagnostic translator converting structured compiler errors into plain-English explanations and remediation steps.

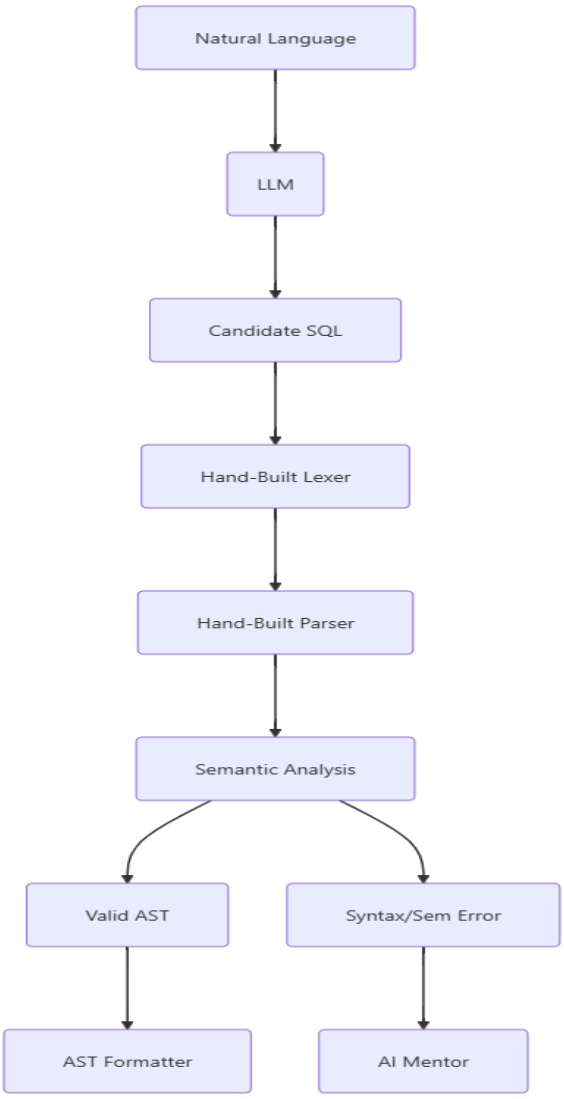
The central design principle is that **AI proposes and the compiler decides**. Because LLMs frequently generate syntactically plausible queries that reference non-existent schema elements or violate relational constraints, this hand-built deterministic pipeline serves as the strict gatekeeper. This report details the formal grammar, the implementation of each compiler phase, the external AI integration design, the test methodology, and key limitations.

2. Introduction & Motivation

Structured Query Language (SQL) serves as the industry standard for relational data retrieval and is the primary target for modern natural-language interfaces. However, generating SQL via generative AI introduces substantial reliability challenges: LLMs can hallucinate non-existent tables, invent phantom columns, execute invalid joins, or produce type mismatches.

This project addresses this issue through a compiler-first architectural pattern:

Fig1: System Flow Diagram



The motivation is grounded in two primary objectives:

1. **Rigorous Compiler Theory:** SQL has an accessible yet sufficiently expressive context-free grammar that exercises every foundational compiler phase — tokenization, recursive-descent parsing, AST generation, symbol table scoping, type consistency verification, and AST unparsing (synthesis).
2. **Defensive AI Engineering:** Rather than trusting an LLM to police its own output through iterative prompting, a deterministic compiler pipeline acts as an authoritative, zero-hallucination verification layer.

The AI components remain strictly modular. If all external model calls are disabled, the system operates as a self-contained, complete compiler pipeline for an analytical SQL subset.

3. Literature Review

The project synthesizes established principles from four key domains of computer science literature:

3.1 Compiler and Parsing Theory

The pipeline follows the classical multi-pass architecture articulated by Aho, Lam, Sethi, and Ullman (2006) in *Compilers: Principles, Techniques, and Tools*. Recursive-descent parsing was specifically chosen over automated parser generators (such as ANTLR, Flex, or Bison) to ensure that the grammar's production rules maintain an explicit 1:1 procedural correspondence in the codebase, fulfilling the pedagogical objectives of compiler construction.

3.2 Natural-Language-to-SQL Generation

Natural-language interface design builds upon benchmarks such as **Spider** (Yu et al., 2018) and **BIRD** (Li et al., 2024). While modern text-to-SQL systems have transitioned from specialized Seq2Seq architectures toward prompted general-purpose LLMs (Gao et al., 2024), these benchmarks demonstrate that prompt-engineered queries consistently suffer from schema-linking degradation when presented with unfamiliar enterprise schemas.

3.3 LLM Hallucination & Deterministic Verification

Recent studies demonstrate that language model hallucinations in SQL query and code generation fall into two broad buckets:

- **Schema-linking errors:** Inaccurate table, column, or foreign key bindings.
- **Logical-synthesis errors:** Flawed condition groupings or invalid aggregate selections.

Research highlights that deterministic AST-based validation catches schema-linking hallucinations with 100% precision without the latency, financial cost, or non-deterministic drift of multi-turn LLM self-reflection.

3.4 Compiler Diagnostic Ergonomics

Compiler error readability directly governs developer remediation speed. Nienaltowski, Pedroni, and Meyer (2008) demonstrated that raw error density often hinders comprehension, while Becker et al. (2021) identified that modular separation between the error cause and the fix significantly lowers cognitive load. This project incorporates findings from Taylor et al. (2023) (*Dcc --help*), utilizing an LLM solely to convert deterministic diagnostic data objects into human-readable explanations without altering the underlying compiler error coordinates.

4. Scope and SQL Subset Definition

The decided grammar is scoped to an expressive subset of ANSI SQL analytical queries.

4.1 In Scope

- SELECT projections with explicit column lists, column aliases (both implicit and explicit AS), and the * wildcard.
- FROM clause referencing a base table with optional table aliases.
- JOIN operations (INNER JOIN and LEFT JOIN) with explicit ON join predicates.
- WHERE clauses supporting relational comparisons (=, !=, <, >, <=, >=), boolean operations (AND, OR), and nested parentheses for explicit precedence.
- Aggregation functions: COUNT(*), COUNT(col), SUM(col), AVG(col), MIN(col), and MAX(col).
- GROUP BY and HAVING clauses.
- ORDER BY clauses with optional ASC and DESC qualifiers.
- LIMIT row restrictions.

4.2 Explicitly Out of Scope

- Nested subqueries and correlated derived tables.
- Set operations (UNION, INTERSECT, EXCEPT).
- Analytical window functions (OVER, PARTITION BY).
- DML mutations (INSERT, UPDATE, DELETE) and DDL schema modifications (CREATE, ALTER, DROP).

5. Formal Grammar Specification (EBNF)

The formal language grammar uses Extended Backus-Naur Form (EBNF), where [...] denotes optional constructs, { ... }* denotes zero or more repetitions, and | indicates alternation.

```
query   ::= select_stmt [ ";" ]

select_stmt    ::= "SELECT" select_list
                   "FROM" table_ref { join_clause }*
                   [ "WHERE" condition ]
                   [ "GROUP" "BY" column_list ]
                   [ "HAVING" condition ]
                   [ "ORDER" "BY" order_list ]
                   [ "LIMIT" INTEGER ]

select_list    ::= "*" | select_item { "," select_item }*
select_item    ::= ( agg_expr | column ) [ alias_clause ]
```

```
agg_expr      ::= "COUNT" "(" ( column | "*" ) ")"
                | ( "SUM" | "AVG" | "MIN" | "MAX" ) "(" column ")"

table_ref     ::= IDENTIFIER [ alias_clause ]
join_clause   ::= [ "INNER" | "LEFT" ] "JOIN" table_ref "ON" condition

condition     ::= or_expr
or_expr       ::= and_expr { "OR" and_expr }*
and_expr      ::= comparison { "AND" comparison }*
comparison    ::= operand comp_op operand | "(" condition ")"
comp_op       ::= "=" | "!=" | "<" | ">" | "<=" | ">="
operand       ::= column | literal

column_list   ::= column { "," column }*
order_list    ::= order_item { "," order_item }*
order_item    ::= column [ "ASC" | "DESC" ]

column        ::= [ IDENTIFIER "." ] IDENTIFIER
alias_clause  ::= [ "AS" ] IDENTIFIER
literal       ::= STRING | NUMBER
```

5.1 Grammar Design Justifications

1. **Precedence Layering:** By nesting condition -> or_expr -> and_expr -> comparison, the grammar enforces that boolean AND naturally binds tighter than OR without requiring external operator precedence declarations.
2. **Deterministic Aggregations:** The grammar permits * strictly inside COUNT(*), rejecting illegal expressions such as SUM(*) at the parsing phase.
3. **Flexible Aliasing:** The rule alias_clause ::= ["AS"] IDENTIFIER uniformly allows both formal aliases (customers AS c) and idiomatic SQL shorthands (customers c).
4. **Resilience to Whitespace in Keywords:** Separating multi-word clauses into distinct tokens (GROUP + BY and ORDER + BY) avoids tokenizer crashes when multi-line whitespace or comments separate words.

6. Lexical Analysis (Lexer)

The lexer scans the raw SQL source string from left to right, identifying terminal tokens and preserving line and column indices for precise diagnostics.

6.1 Token Categories

Token Type	Representation / Examples	Description
KEYWORD	SELECT, FROM, WHERE, JOIN, INNER, LEFT, ON, GROUP, ORDER, BY, HAVING, LIMIT, AND, OR, AS, ASC, DESC	Case-insensitive matching against a reserved keyword table.
IDENTIFIER	customers, order_date, id	Names of tables, columns, or alias labels.
AGG_FUNC	COUNT, SUM, AVG, MIN, MAX	Aggregate function identifiers.
NUMBER	42, 3.14159	Unsigned integer and floating-point numeric constants.
STRING	'Delhi', 'Active'	Single-quote delimited character sequences.
OPERATOR	=, !=, <, >, <=, >=	Relational comparison tokens.
PUNCTUATION	(,), ,, ., *	Delimiters, qualification operators, and projection wildcards.
EOF	\0	Explicit terminal token signaling end of input.

6.2 Lexical Rules & Behavior

- **Case Normalization:** Keywords are converted to upper-case tokens internally, permitting flexible case entry (select, Select, SELECT).
- **Trivia Discarding:** Standard whitespace, line breaks, and SQL single-line comments (-- comment) are skipped.

- **Lexical Error Handling:** Any character outside the supported character set (e.g., #, @, $) triggers an immediate LexicalError with accurate file coordinates.

7. Syntax Analysis (Parser) & AST Design

The parser implements a predictive, top-down recursive-descent approach with an LL(1) lookahead window. Each non-terminal symbol in the grammar maps directly to a parsing routine.

7.1 AST Node Hierarchy

The parser produces a strongly typed Abstract Syntax Tree:

```
ASTNode (base)
 ├── SelectStmtNode(
 │     projections: List[SelectItemNode],
 │     from_table: TableRefNode,
 │     joins: List[JoinNode],
 │     where: Optional[ConditionNode],
 │     group_by: Optional[List[ColumnNode]],
 │     having: Optional[ConditionNode],
 │     order_by: Optional[List[OrderItemNode]],
 │     limit: Optional[int]
 │   )
 ├── SelectItemNode(expr: Union[ColumnNode, AggExprNode], alias: Optional[str])
 ├── TableRefNode(name: str, alias: Optional[str])
 ├── JoinNode(join_type: str, table: TableRefNode, on_condition: ConditionNode)
 ├── AggExprNode(func: str, argument: Union[ColumnNode, str])
 ├── BinaryConditionNode(op: str, left: ConditionNode, right: ConditionNode)
 ├── ComparisonNode(op: str, left: OperandNode, right: OperandNode)
 ├── ColumnNode(table_prefix: Optional[str], column_name: str)
 └── LiteralNode(value: Union[str, int, float], data_type: str)
```

7.2 Parse Tree Visualization

For the input query:

```
SELECT name, city FROM customers WHERE city = 'Delhi' AND signup_date > '2024-01-01'
```

The resulting Abstract Syntax Tree is generated as:

```
SelectStmtNode
├── projections:
│   ├── SelectItemNode -> ColumnNode(customers.name)
│   └── SelectItemNode -> ColumnNode(customers.city)
├── from_table: TableRefNode(customers)
├── joins: []
├── where: BinaryConditionNode(AND)
│   ├── left: ComparisonNode(=)
│   │   ├── left: ColumnNode(city)
│   │   └── right: LiteralNode('Delhi', VARCHAR)
│   └── right: ComparisonNode(>)
│       ├── left: ColumnNode(signup_date)
│       └── right: LiteralNode('2024-01-01', DATE)
├── group_by: null
├── having: null
├── order_by: null
└── limit: null
```

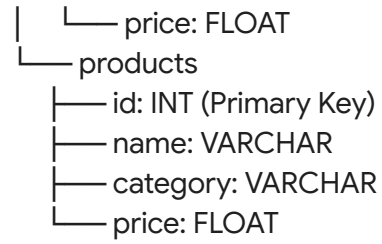
8. Semantic Analysis & Symbol Table

Syntactic correctness does not ensure query validity. The semantic analyzer walks the AST and validates identifiers and expressions against a relational symbol table (Database Catalog).

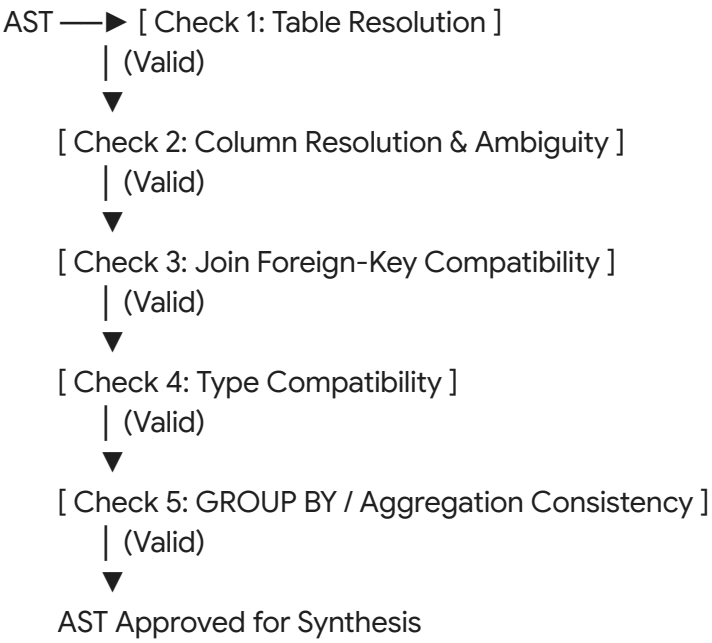
8.1 Symbol Table Architecture

The schema catalog maintains the reference environment for validation:

```
DatabaseCatalog
├── customers
│   ├── id: INT (Primary Key)
│   ├── name: VARCHAR
│   ├── city: VARCHAR
│   └── signup_date: DATE
├── orders
│   ├── id: INT (Primary Key)
│   ├── customer_id: INT (Foreign Key -> customers.id)
│   ├── order_date: DATE
│   └── status: VARCHAR
├── order_items
│   ├── order_id: INT (Foreign Key -> orders.id)
│   ├── product_id: INT (Foreign Key -> products.id)
│   ├── quantity: INT
```



8.2 Semantic Validation Checks



8.3 Properties

1. **Table Resolution:** Every identifier in FROM and JOIN must match an active table in the catalog or declare a unique alias.
2. **Column Scope & Ambiguity Checking:** Columns must belong to tables in the current scope. If a query joins multiple tables and references an unqualified column (e.g., id), the analyzer inspects all joined tables and flags ambiguity if id exists in more than one table.
3. **Type Compatibility:** The types of left and right operands in a ComparisonNode must match or have safe implicit promotions. Comparing a DATE column to an INT literal raises a semantic failure.
4. **Join Relationship Verification:** Column pairs used in JOIN ... ON conditions are verified against catalog foreign key declarations to catch accidental Cartesian products or logically invalid joins.
5. **Aggregation Scope Consistency:** When GROUP BY is present, every column in the SELECT list that is not enclosed inside an aggregate function (COUNT, SUM, etc.) must appear explicitly in the GROUP BY list.

9. Synthesis Phase (AST Formatter)

The synthesis phase represents the compiler's code-generation step. The formatter performs a structured AST traversal and reconstructs canonical SQL text.

9.1 Synthesis Rules

- Reserved keywords are normalized to uppercase.
- Major clauses (SELECT, FROM, JOIN, WHERE, GROUP BY, ORDER BY, LIMIT) break onto fresh lines.
- Projections and condition expressions are indented by two spaces.
- Multi-clause conditions mirror tree parenthesization, making logical precedence obvious.

9.2 Synthesis Example

Input Query:

select c.name, count(*) from customers c join orders o on c.id=o.customer_id where c.city='Delhi' group by c.name having count(*)>2 order by c.name asc limit 10;

Synthesized Canonical Output:

```
SELECT
  c.name,
  COUNT(*)
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE c.city = 'Delhi'
GROUP BY c.name
HAVING COUNT(*) > 2
ORDER BY c.name ASC
LIMIT 10;
```

10. AI Integration Architecture

AI models are positioned entirely outside the compiler pipeline, serving at the outer input and output boundaries.

10.1 Principle: AI Proposes, Compiler Decides

1. **Input Edge (NL-to-SQL):** A natural language request is converted into candidate SQL via an LLM. The candidate query is treated as completely untrusted input and submitted to the lexer identically to human-written text.

2. **Output Edge (AI Error Mentor):** When the compiler pipeline encounters an error (lexical, syntactic, or semantic), it generates a structured DiagnosticRecord. This record is passed to an LLM to generate plain-English remediation suggestions. The original deterministic error message is preserved alongside the AI explanation.

10.2 Structured Diagnostic Exchange

When an invalid query is processed:


```
{
  "phase": "SEMANTIC_ANALYSIS",
  "error_code": "ERR_COLUMN_NOT_FOUND",
  "message": "Column 'full_name' does not exist in table 'customers'.",
  "line": 1,
  "column": 8,
  "context": "SELECT full_name FROM customers",
  "available_columns": ["id", "name", "city", "signup_date"]
}
```


The AI explanation layer processes this record and outputs: *"The query tries to select a column named full_name, but the customers table only has id, name, city, and signup_date. To fix this, change full_name to name."*

11. Test Suite & Validation Matrix

The system is evaluated against seven distinct testing scenarios:

#	Test Category	Sample Query	Verification Point	Expected Outcome
1	Simple Projection	SELECT * FROM customers;	Wildcard resolution, single-table parsing	Passes all phases; formats cleanly.
2	Boolean Precedence	SELECT name FROM customers WHERE city = 'Delhi' AND signup_date > '2024-01-01'	Parsing of nested AND / OR expressions	Produces properly layered binary condition tree.

3	Join Resolution	SELECT c.name, o.status FROM customers c JOIN orders o ON c.id = o.customer_id	Implicit aliases, foreign key relationship validation	Passes table and column binding; formats cleanly.
4	Group By Validation	SELECT city, COUNT(*) FROM customers GROUP BY city HAVING COUNT(*) > 5	Aggregate and non-aggregate column tracking	Validates that city is in GROUP BY; permits COUNT(*).
5	Syntax Error Catch	SELECT name customers WHERE id = 1	Missing FROM keyword	Lexer succeeds; parser halts at token customers (Line 1, Col 13).
6	Semantic Type Mismatch	SELECT name FROM customers WHERE signup_date = 42	Type verification across relational operators	Parser succeeds; semantic analyzer flags DATE vs INT.
7	Hallucination Detection	SELECT full_name, order_count FROM customers	Plausible LLM hallucination catching	Parser succeeds; validator rejects nonexistent columns.

12. Limitations & Future Work

12.1 Current Limitations

- **Grammar Scope:** The grammar excludes subqueries, set operators (UNION), and window functions (OVER).
- **Catalog Scale:** The catalog is designed around a clean, normalized relational model and has not been tested against large legacy databases with hundreds of columns.
- **Dialect Support:** The system targets an ANSI SQL subset and does not support vendor-specific syntax (e.g., T-SQL TOP or PostgreSQL ILIKE).

12.2 Future Work

- **Intermediate Representation (IR):** Translate validated ASTs into Relational Algebra execution trees.
- **Query Optimization:** Implement heuristic optimization passes, such as predicate pushdown and redundant projection elimination.
- **Dialect Transpilation:** Unparse the validated AST into dialect-specific SQL targets (e.g., PostgreSQL, SQLite, BigQuery).

13. Novelty

1. **Deterministic Schema Linking:** Standard LLMs frequently hallucinate column names that sound correct but don't exist (e.g., using full_name instead of name). Our semantic validator resolves identifiers against the schema symbol table during AST traversal, catching schema-linking errors deterministically before the query reaches the database.
2. **Context-Aware Error Translation:** Compiler diagnostic messages are traditionally cryptic. Rather than letting an LLM parse the error text blindly, our compiler exports a strongly typed diagnostic object (Node, Line, Col, Expected, Found) directly into a prompt template, generating plain-English remediation without losing technical accuracy.
3. **AST-Based Auto-Correction (Fuzzy Catalog Matching):** When the semantic analyzer catches an invalid column or table name (such as cust_id instead of customer_id), compute the Levenshtein distance against valid identifiers in the symbol table. If distance <= 2, have the compiler automatically repair the AST node and output a notice: "Auto-corrected cust_id -> customer_id."

14. References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley / Pearson.
2. Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (2021). On designing programming error messages for novices: Readability and its constituent factors. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–14.
3. Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2024). A survey on employing large language models for text-to-SQL tasks. *arXiv preprint arXiv:2407.15186*.

4. Li, J., Hui, B., Qu, G., Yang, J., Li, B., Wang, B., ... & Li, Y. (2024). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQL evaluation (BIRD). *Advances in Neural Information Processing Systems (NeurIPS 2024)*.
5. Nienaltowski, M.-H., Pedroni, M., & Meyer, B. (2008). Compiler error messages: What can help novices? *ACM SIGCSE Bulletin*, 40(1), 168–172.
6. Taylor, A., Grover, S., & Srikant, S. (2023). Dcc --help: Generating context-aware compiler error explanations with large language models. *arXiv preprint arXiv:2308.11873*.
7. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., ... & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3911–3921.